

Cómo probar la Etapa 7 paso a paso

La Etapa 7 se prueba en dos niveles:

1. Desde la UI, para validar filtros por documento, fuentes y previews.
2. Desde API/curl, para validar ranking, chunks usados, scores y evaluaciones RAG.

La parte clave a verificar es que el flujo ahora pase `documentIds` al chat, que el backend use esos filtros en `SearchAsync`, y que cada fuente devuelva `score`, `vectorScore`, `keywordScore`, `rankScore` y `preview`. El contrato frontend envía `documentIds` en `sendMessage`. El backend recibe esos IDs en `ChatController` y los pasa a `GenerateResponseAsync`. Luego `RagService.SearchAsync` filtra chunks por documento, aplica umbral, rankea y devuelve los mejores chunks.

1. Preparar el entorno

1.1. Levantar Ollama

Primero asegurate de tener Ollama corriendo y los modelos descargados. El setup del proyecto indica usar `qwen2.5-coder:7b` y `nomic-embed-text`.

```
ollama pull qwen2.5-coder:7b
```

```
ollama pull nomic-embed-text
```

```
ollama serve
```

Si querés usar el modelo liviano que aparece en `appsettings.Development.json`, también podés tener `llama3.2:1b`, porque en desarrollo está configurado como modelo de chat.

```
ollama pull llama3.2:1b
```

1.2. Revisar configuración RAG

En `appsettings.json`, RAG está configurado con:

- `ChunkSize: 900`
- `ChunkOverlap: 150`
- `MaxRetrievedChunks: 4`
- `MinSimilarityScore: 0.2`
- `VectorStoreProvider: Local`

Esto es lo ideal para probar varias fuentes.

Pero ojo: en `appsettings.Development.json`, `MaxRetrievedChunks` está en `1`, por lo que en desarrollo probablemente veas una sola fuente aunque haya más chunks relevantes.

Para probar bien Etapa 7, te recomiendo levantar el backend sobrescribiendo estos valores:

```
Rag__MaxRetrievedChunks=4 Rag__MinSimilarityScore=0.15 dotnet run --urls  
http://localhost:5201
```

2. Levantar backend y frontend

2.1. Backend

Desde la raíz del repo:

```
cd backend/LocalMind.Api
```

```
dotnet restore
```

```
dotnet ef database update
```

```
Rag__MaxRetrievedChunks=4 Rag__MinSimilarityScore=0.15 dotnet run --urls  
http://localhost:5201
```

El setup del proyecto ya documenta el flujo base de backend con `dotnet restore`, migración y `dotnet run`.

2.2. Frontend

En otra terminal:

```
cd frontend
```

```
npm install
```

```
VITE_API_BASE_URL=http://localhost:5201/api npm run dev
```

El frontend tiene scripts `dev` y `build` definidos en `package.json`. Además, el cliente Axios usa `VITE_API_BASE_URL` o, por defecto, `http://localhost:5201/api`.

Abrí:

```
http://localhost:5173
```

3. Prueba manual desde la UI

3.1. Registrarte o iniciar sesión

Entrá a:

```
http://localhost:5173/register
```

o:

```
http://localhost:5173/login
```

El backend expone `/api/auth/register` y `/api/auth/login`.

3.2. Subir documentos de prueba

Para probar bien los filtros, conviene subir dos documentos con temas distintos.

Ejemplo 1: `rag-qdrant.md`

```
# Arquitectura RAG
```

El sistema usa recuperación aumentada con generación para responder preguntas

sobre documentos. El ranking combina similitud vectorial y coincidencia de

palabras clave. Qdrant puede usarse como vector store real con Docker.

Las fuentes deben mostrar el documento, el chunk usado y un preview del texto.

Ejemplo 2: `seguridad.md`

```
# Seguridad del sistema
```

El sistema protege rutas con JWT. También valida mensajes para evitar intentos

de prompt injection. Las métricas registran tiempo de respuesta, ruta usada,

uso de herramientas y cantidad de chunks RAG utilizados.

Subilos desde la sección Documentos RAG. El frontend llama a `uploadDocument(file)` contra `/documents/upload`. El backend recibe ese archivo en `DocumentsController.Upload`.

Qué deberías ver

Después de subir cada documento:

- Aparece en la lista.
- Muestra cantidad de chunks.
- Muestra tamaño.
- Muestra estado `Processed`.

La UI muestra esos datos desde la lista de documentos.

4. Probar RAG sin filtro de documento

En el chat, preguntá algo así:

Según el documento, ¿qué dice sobre Qdrant y el ranking RAG?

Importante: incluí palabras como documento, fuente, archivo, RAG o similares. El detector considera una pregunta como documental si contiene términos como `documento`, `archivo`, `pdf`, `fuente`, `según el documento`, `contexto cargado` o `rag`.

Resultado esperado

Deberías ver:

- Respuesta del asistente.
- Badge tipo RAG con fuentes.
- Sección Fuentes RAG usadas.
- Cada fuente con:

```

doc:X#chunk:Y
rank
Vector
Keywords
Final

```

La UI renderiza estas fuentes y métricas desde `item.sources`.

5. Probar filtro por documento

5.1. Seleccionar solo `seguridad.md`

En la lista lateral de documentos:

1. Marcá solamente `seguridad.md`.
2. Dejá desmarcado `rag-qdrant.md`.
3. Preguntá:

Según el documento, ¿qué dice sobre JWT y seguridad?

Resultado esperado

Las fuentes deberían venir solo del documento seleccionado. El frontend mantiene `selectedDocumentIds` y los envía como `documentIds` al backend cuando mandás el mensaje.

El backend filtra los chunks por esos IDs antes de rankear.

5.2. Seleccionar solo `rag-qdrant.md`

Ahora marcá solamente `rag-qdrant.md` y preguntá:

Según el documento, ¿qué dice sobre Qdrant?

Resultado esperado

Deberías ver fuentes de `rag-qdrant.md`.

5.3. Probar una pregunta cruzada con filtro incorrecto

Marcá solamente `seguridad.md` y preguntá:

Según el documento, ¿qué dice sobre Qdrant?

Resultado esperado

Puede pasar una de estas dos cosas:

- Que no haya contexto suficiente y el sistema responda sin RAG.
- Que devuelva baja relevancia o ninguna fuente, dependiendo del `MinSimilarityScore`.

Esto valida que el filtro realmente está limitando el universo de búsqueda.

6. Probar ranking híbrido

El ranking ahora combina:

`score final = 75% vector + 25% keywords`

La lógica está en `CalculateRankScore`. Además, el match se construye con `vectorScore`, `keywordScore`, `rankScore` y `preview`.

Prueba sugerida

Preguntá:

Según el documento RAG, ¿cómo se usa Qdrant para vector store?

Qué mirar

En fuentes:

- Si el chunk contiene explícitamente `qdrant`, debería tener mejor `keywordScore`.
- Si el chunk es semánticamente parecido aunque no tenga todas las palabras, debería aportar por `vectorScore`.
- El `Final` debería coincidir con el valor combinado.
- El orden debería priorizar el mayor `rank`.

7. Probar previews de chunks

Hacé cualquier pregunta documental que active RAG, por ejemplo:

Según el documento, ¿qué fuentes usa el sistema para responder?

Después abrí:

Ver preview del chunk usado

Resultado esperado

Deberías ver un extracto corto del chunk usado como fuente. El backend genera el preview en `BuildPreview` y lo incluye dentro de `RagChunkMatch`.

8. Probar por API con `curl`

Esto es útil para verificar valores exactos sin depender de la UI.

8.1. Crear usuario y obtener token

```
TOKEN=$(  
  
curl -s -X POST http://localhost:5201/api/auth/register \  
  
-H "Content-Type: application/json" \  
  
-d '{"email":"etapa7@test.com","password":"Password123!"}' \  
  
| jq -r '.token'  
  
)  
  
echo "$TOKEN"
```

Si el usuario ya existe, usá login:

```
TOKEN=$(
curl -s -X POST http://localhost:5201/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"etapa7@test.com","password":"Password123!"}' \
| jq -r '.token'
)

echo "$TOKEN"
```

Los endpoints de auth son /api/auth/register y /api/auth/login.

8.2. Crear documentos de prueba por terminal

```
mkdir -p /tmp/localmind-rag-test
```

```
cat > /tmp/localmind-rag-test/rag-qdrant.md <<'EOF'
```

```
# Arquitectura RAG
```

```
El sistema usa recuperación aumentada con generación para responder preguntas
```

```
sobre documentos. El ranking combina similitud vectorial y coincidencia de
```

```
palabras clave. Qdrant puede usarse como vector store real con Docker.
```

```
Las fuentes deben mostrar el documento, el chunk usado y un preview del texto.
```

```
EOF
```

```
cat > /tmp/localmind-rag-test/seguridad.md <<'EOF'
```

```
# Seguridad del sistema
```

```
El sistema protege rutas con JWT. También valida mensajes para evitar intentos
```

```
de prompt injection. Las métricas registran tiempo de respuesta, ruta usada,
```

```
uso de herramientas y cantidad de chunks RAG utilizados.
```

```
EOF
```

8.3. Subir documentos

```
DOC_RAG=$(
```

```
curl -s -X POST http://localhost:5201/api/documents/upload \
```

```
-H "Authorization: Bearer $TOKEN" \
```

```
-F "file=@/tmp/localmind-rag-test/rag-qdrant.md" \
```

```
| jq -r '.id'
```

```
)
```

```
DOC_SEGURIDAD=$(
```

```
curl -s -X POST http://localhost:5201/api/documents/upload \
```

```
-H "Authorization: Bearer $TOKEN" \
```

```
-F "file=@/tmp/localmind-rag-test/seguridad.md" \  
  
| jq -r '.id' \  
  
) \  
  
echo "DOC_RAG=$DOC_RAG" \  
  
echo "DOC_SEGURIDAD=$DOC_SEGURIDAD" \  
  
El upload está expuesto en POST /api/documents/upload.
```

8.4. Listar documentos indexados

```
curl -s http://localhost:5201/api/documents \  
  
-H "Authorization: Bearer $TOKEN" \  
  
| jq \  
  
El listado usa GET /api/documents.
```

8.5. Ver chunks de un documento

```
curl -s http://localhost:5201/api/documents/$DOC_RAG/chunks \  
  
-H "Authorization: Bearer $TOKEN" \  
  
| jq \  
  
El endpoint de chunks es GET /api/documents/{documentId}/chunks.
```

9. Probar chat RAG por API

9.1. Consulta RAG sobre todos los documentos

```
curl -s -X POST http://localhost:5201/api/chat/send \  
  
-H "Authorization: Bearer $TOKEN" \  
  
| jq
```

```
-H "Content-Type: application/json" \  
  
-d "{  
  
  \"conversationId\": null,  
  
  \"message\": \"Según el documento, ¿qué dice sobre Qdrant y ranking RAG?\",  
  
  \"documentIds\": []  
  
}" \  
  
| jq '{  
  
  conversationId,  
  
  usedRag,  
  
  route,  
  
  chunksUsed,  
  
  sources  
  
}'
```

El chat se envía a `POST /api/chat/send`.

Resultado esperado

Esperá algo parecido a:

```
{  
  
  "usedRag": true,  
  
  "route": "rag",  
  
  "chunksUsed": 1,  
  
  "sources": [  
  
  ]  
  
}
```

```
{
  "documentId": 1,
  "fileName": "rag-qdrant.md",
  "chunkIndex": 0,
  "score": 0.72,
  "vectorScore": 0.68,
  "keywordScore": 0.85,
  "rankScore": 0.72,
  "preview": "Arquitectura RAG ...",
  "chunkReference": "doc:1#chunk:0"
}
```

Los valores exactos cambian según embeddings/modelo/documento, pero deberían aparecer esos campos porque el frontend y backend ya esperan `score`, `vectorScore`, `keywordScore`, `rankScore` y `preview`.

9.2. Consulta filtrando solo un documento

```
curl -s -X POST http://localhost:5201/api/chat/send \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d "{
```

```
\ "conversationId\": null,  
  
  \ "message\": \ "Según el documento, ¿qué dice sobre JWT y seguridad?\",  
  
  \ "documentIds\": [ ${DOC_SEGURIDAD} ]  
  
}" \
```

```
| jq '{  
  
  usedRag,  
  
  route,  
  
  chunksUsed,  
  
  sources: [ .sources[] | {  
  
    documentId,  
  
    fileName,  
  
    chunkIndex,  
  
    score,  
  
    vectorScore,  
  
    keywordScore,  
  
    rankScore,  
  
    preview  
  
  } ]  
  
'
```

Resultado esperado

Todas las fuentes deberían tener:

```
"documentId": DOC_SEGURIDAD
```

Eso confirma que el filtro por documento está funcionando. El servicio RAG hace el `Where` por `documentIds.Contains(chunk.DocumentId)`.

10. Probar evaluación RAG

La Etapa 7 incluye un endpoint de evaluación:

```
POST /api/documents/rag/evaluate
```

Está en `DocumentsController`.

10.1. Evaluación básica

```
curl -s -X POST http://localhost:5201/api/documents/rag/evaluate \

-H "Authorization: Bearer $TOKEN" \

-H "Content-Type: application/json" \

-d "[

{

  \"query\": \"¿Qué dice el documento sobre Qdrant?\",

  \"expectedDocumentIds\": [$DOC_RAG],

  \"documentIds\": [$DOC_RAG],

  \"minSimilarityScore\": 0.15,

  \"maxRetrievedChunks\": 4

},

{

  \"query\": \"¿Qué dice el documento sobre JWT?\",
```

```

    \"expectedDocumentIds\": [${DOC_SEGURIDAD}],

    \"documentIds\": [${DOC_SEGURIDAD}],

    \"minSimilarityScore\": 0.15,

    \"maxRetrievedChunks\": 4

  }

]\" \\

| jq

```

Qué revisar

La respuesta debería incluir:

- totalQuestions
- questionsWithContext
- expectedHits
- contextCoverage
- expectedHitRate
- averageTopScore
- items

La implementación de `EvaluateAsync` calcula esos valores usando búsquedas RAG internas.

10.2. Probar umbral bajo vs alto

Umbral permisivo

```

curl -s -X POST http://localhost:5201/api/documents/rag/evaluate \

-H "Authorization: Bearer $TOKEN" \

-H "Content-Type: application/json" \

-d "[

{

  \"query\": \"¿Qué dice el documento sobre Qdrant?\",

```

```

        \"expectedDocumentIds\": [${DOC_RAG}],

        \"minSimilarityScore\": 0.05,

        \"maxRetrievedChunks\": 4

    }

]\" \\

| jq '{contextCoverage, expectedHitRate, averageTopScore, items}'

```

Umbral exigente

```

curl -s -X POST http://localhost:5201/api/documents/rag/evaluate \

-H "Authorization: Bearer $TOKEN" \

-H "Content-Type: application/json" \

-d "[

{

    \"query\": \"¿Qué dice el documento sobre Qdrant?\",

    \"expectedDocumentIds\": [${DOC_RAG}],

    \"minSimilarityScore\": 0.90,

    \"maxRetrievedChunks\": 4

}

]\" \\

| jq '{contextCoverage, expectedHitRate, averageTopScore, items}'

```

Resultado esperado

Con `0.05` deberías recuperar más contexto.

Con `0.90` probablemente recuperes poco o nada, dependiendo del embedding.

Esto valida el ajuste de umbrales de similitud. El umbral se toma desde `RagSearchOptions.MinSimilarityScore` o desde configuración si no se manda.

11. Checklist de aceptación de Etapa 7

Usá esta lista para confirmar que está todo OK.

Backend

- Puedo subir `.md`, `.txt` o `.pdf`.
- Se crean chunks.
- Se generan embeddings sin error.
- `GET /api/documents` devuelve documentos con `chunkCount`.
- `GET /api/documents/{id}/chunks` devuelve chunks.
- `POST /api/chat/send` con pregunta documental devuelve `usedRag: true`.
- La respuesta trae `sources`.
- Cada fuente trae `score`.
- Cada fuente trae `vectorScore`.
- Cada fuente trae `keywordScore`.
- Cada fuente trae `rankScore`.
- Cada fuente trae `preview`.
- El filtro `documentIds` limita las fuentes.
- `POST /api/documents/rag/evaluate` devuelve métricas de evaluación.

Frontend

- Puedo subir documentos desde la UI.
- Puedo marcar/desmarcar documentos.
- El texto “Filtro RAG” cambia según la selección.
- Al preguntar, se envían los documentos seleccionados.
- La respuesta muestra “Fuentes RAG usadas”.
- Puedo ver scores vector/keywords/final.
- Puedo abrir el preview del chunk.
- Si limpio el filtro, busca en todos los documentos.

12. Problemas comunes

12.1. No aparece RAG, aparece chat normal

Probablemente la pregunta no activó `IsDocumentQuestion`.

Usá frases como:

Según el documento...

En mis documentos...

Usando las fuentes RAG...

El detector busca palabras como documento, archivo, pdf, fuente, según el documento, contexto cargado y rag.

12.2. Solo aparece una fuente

En desarrollo, `MaxRetrievedChunks` está configurado en 1.

Solución:

```
Rag__MaxRetrievedChunks=4 Rag__MinSimilarityScore=0.15 dotnet run --urls  
http://localhost:5201
```

12.3. No hay fuentes aunque subí documentos

Revisá:

```
curl -s http://localhost:5201/api/documents \
-H "Authorization: Bearer $TOKEN" \
| jq
```

Si `chunkCount` es 0 o no aparece el documento, falló la indexación.

También podés bajar temporalmente el umbral:

```
Rag__MinSimilarityScore=0.05 dotnet run --urls http://localhost:5201
```

12.4. Ollama tarda mucho

Usá modelo chico en desarrollo:

```
"Model": "llama3.2:1b"
```

En `appsettings.Development.json` ya está configurado así.

12.5. Al reabrir una conversación no veo las fuentes

Con el código actual, el historial recupera `role`, `content` y `createdAt`, pero no persiste las fuentes RAG en la tabla de mensajes. El endpoint de historial arma mensajes con esos campos.

Entonces las fuentes se ven en la respuesta recién generada, pero no necesariamente al reabrir una conversación antigua. Para persistirlas habría que guardar metadata RAG por mensaje.

13. Orden recomendado de prueba rápida

Si querés probar todo en 10 minutos:

Terminal 1

ollama serve

Terminal 2

cd backend/LocalMind.Api

dotnet ef database update

Rag_MaxRetrievedChunks=4 Rag_MinSimilarityScore=0.15 dotnet run --urls

http://localhost:5201

Terminal 3

cd frontend

VITE_API_BASE_URL=http://localhost:5201/api npm run dev

Después:

1. Abrí `http://localhost:5173`.
2. Registrarte.
3. Subí `rag-qdrant.md`.
4. Subí `seguridad.md`.
5. Preguntá: Según el documento, ¿qué dice sobre Qdrant?
6. Verificá fuentes, scores y preview.
7. Marcá solo `seguridad.md`.
8. Preguntá: Según el documento, ¿qué dice sobre JWT?
9. Verificá que la fuente sea solo `seguridad.md`.
10. Probá `/api/documents/rag/evaluate` con `curl`.

14. Comandos que usé para preparar esta guía

-  `git status --short`
-  `nl -ba docker-compose.yml | sed -n '1,220p'`
-  `nl -ba docs/setup.md | sed -n '1,220p'`
-  `nl -ba frontend/package.json | sed -n '1,80p'`
-  `nl -ba backend/LocalMind.Api/Program.cs | sed -n '1,220p'`
-  `nl -ba backend/LocalMind.Api/Controllers/DocumentController.cs | sed -n '1,120p'`

- ✓ nl -ba backend/LocalMind.Api/Controllers/ChatController.cs | sed -n '45,105p'
- ✓ nl -ba backend/LocalMind.Api/Services/Rag/IRagService.cs | sed -n '1,80p'
- ✓ nl -ba frontend/src/features/chat/chatService.ts | sed -n '1,120p'
- ✓ nl -ba frontend/src/features/documents/documentService.ts | sed -n '1,80p'
- ✓ nl -ba backend/LocalMind.Api/Controllers/AuthController.cs | sed -n '1,140p'
- ✓ nl -ba frontend/src/features/chat/ChatPage.tsx | sed -n '187,320p;423,520p;563,620p'
- ✓ nl -ba backend/LocalMind.Api/Services/Rag/RagService.cs | sed -n '219,351p'
- ✓ nl -ba backend/LocalMind.Api/appsettings.Development.json | sed -n '1,120p'
- ✓ nl -ba backend/LocalMind.Api/appsettings.json | sed -n '1,160p'
- ✓ nl -ba frontend/src/services/api.ts | sed -n '1,120p'
- ✓ nl -ba backend/LocalMind.Api/Services/Tools/ToolIntentDetector.cs | sed -n '1,180p'